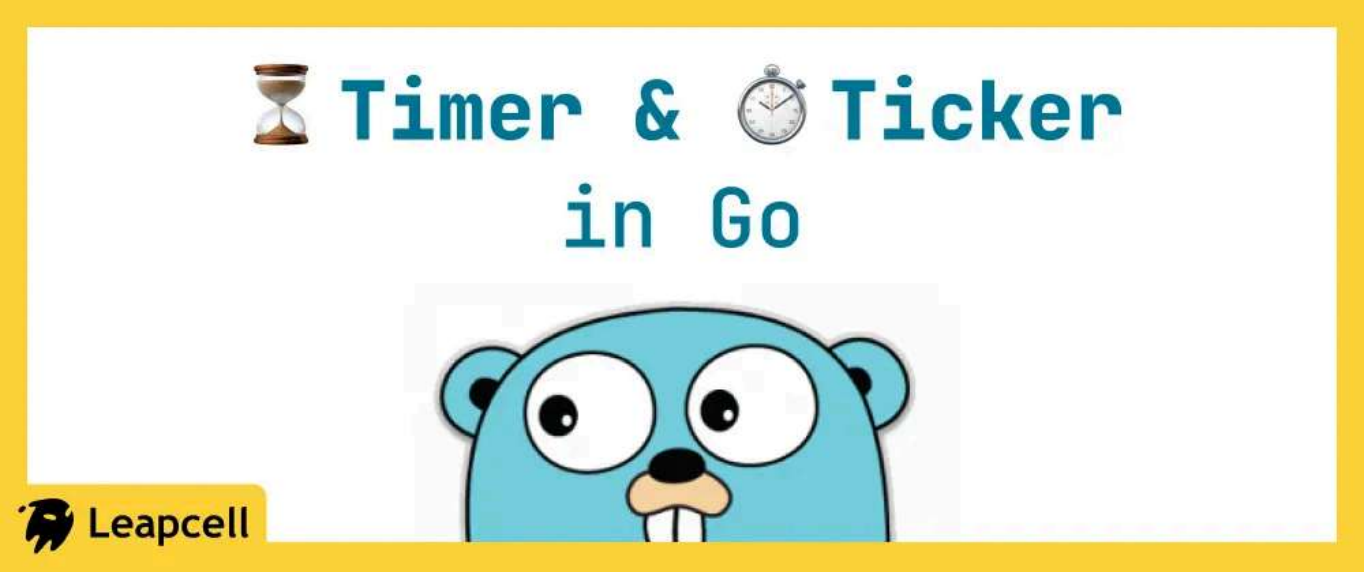
Working with Scheduled Tasks in Go: Timer and Ticker

Explains Go's Timer and Ticker with usage, differences, and resource management tips.

Leapcell Follow 9 min read · Jun 30, 2025



Preface

In daily development, we may encounter situations where we need to delay the execution of some tasks or execute them periodically. At this point, we

need to use timers in Go.

In Go, there are two types of timers: `time.Timer` (one-shot timer) and `time.Ticker` (periodic timer). This article will introduce both types of timers.

Timer: One-Shot Timer

A Timer is a one-shot timer used to perform an operation once at a specific time in the future.

Basic Usage

There are two ways to create a Timer:

- `NewTimer(d Duration) *Timer` : This function accepts a parameter `d` of type `time.Duration` (the time interval), which indicates how long the timer should wait before expiring. `NewTimer` returns a new Timer, which internally maintains a channel `C` . When the timer fires, the current time is sent to channel `C` .
- `AfterFunc(d Duration, f func()) *Timer` : Accepts a specified time interval `d` and a callback function `f` . This function returns a new Timer, and when the timer expires, it directly calls `f` instead of sending a signal through channel `C` . Calling the Timer's `Stop` method can stop the timer and cancel the execution of `f` .

The following code demonstrates how to use `NewTimer` and `AfterFunc` to create timers and their basic usage:

```
package main
```

```
import (
    "fmt"
    "time"
)

func main() {
    // Create a timer using NewTimer
    timer := time.NewTimer(time.Second)
    go func() {
        select {
        case <-timer.C:
            fmt.Println("timer fired!")
        }
    }()
    // Create another timer using AfterFunc
    time.AfterFunc(time.Second, func() {
        fmt.Println("timer2 fired!")
    })
    // Main goroutine waits for 2 seconds to ensure we see the timer output
    time.Sleep(time.Second * 2)
}
```

The output of the above code is as follows:

```
timer fired!
timer2 fired!
```

Here is a step-by-step explanation of the code:

1. Use `NewTimer` to create a timer, then listen to its `C` property in a new goroutine to wait for the timer to fire.
2. Use `AfterFunc` to create another timer, specifying a callback function to handle the timer expiration event.

3. The main goroutine waits long enough to ensure the timer's firing information can be printed.

Method Details

Reset

`Reset(d Duration) bool`: This method is used to reset the expiration time of a Timer, essentially reactivating it. It accepts a parameter `d` of type `time.Duration`, representing how long the timer should wait before expiring.

In addition, this method returns a `bool` value:

- If the timer is active, it returns `true`.
- If the timer has already expired or been stopped, it returns `false` (note: `false` does not mean the reset failed, it only indicates the current state of the timer).

Here is a code example:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    timer := time.NewTimer(5 * time.Second)
    // First reset: the timer is active, so returns true
    b := timer.Reset(1 * time.Second)
    fmt.Println(b) // true

    second := time.Now().Second()
    select {
```

```
    case t := <-timer.C:
        fmt.Println(t.Second() - second) // 1s
    }

    // Second reset: the timer has already expired, so returns false
    b = timer.Reset(2 * time.Second)
    fmt.Println(b) // false
    second = time.Now().Second()

    select {
    case t := <-timer.C:
        fmt.Println(t.Second() - second) // 2s
    }
}
```

The output of the code is as follows:

```
true
1
false
2
```

Step-by-step explanation:

1. Create a timer set to expire after 5 seconds.
2. Call the `Reset` method immediately to set it to expire in 1 second. Since the timer is still active (not expired), `Reset` returns `true`.
3. The `select` statement waits for the timer to expire and prints the actual seconds passed (about 1 second).
4. The timer is reset again, this time to expire in 2 seconds. Since the timer has already expired, `Reset` returns `false`.

5. The `select` statement again waits for the timer to expire and prints the seconds passed (about 2 seconds).

Stop

`Stop() bool` : This method is used to stop the timer. If the timer is successfully stopped, it returns `true` . If the timer has already expired or been stopped, it returns `false` . Note: the `Stop` operation does not close channel `C` .

Here is a code example:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    timer := time.NewTimer(3 * time.Second)
    // Stop the timer before it fires, so returns true
    stop := timer.Stop()
    fmt.Println(stop) // true

    stop = timer.Stop()
    // Stop the timer again, since it is already stopped, returns false
    fmt.Println(stop) // false
}
```

The output is as follows:

```
true
```

```
false
```

Step-by-step explanation:

1. Create a timer set to fire after 3 seconds.
2. Immediately call the `Stop` method to stop the timer. Since the timer has not yet fired, `Stop` returns `true`.
3. Call `Stop` again to try to stop the same timer. Since it is already stopped, this time `Stop` returns `false`.

Ticker: Periodic Timer

A Ticker is a periodic timer used to execute tasks repeatedly at fixed intervals. At every interval, it sends the current time to its channel.

Basic Usage

We can use the `NewTicker` function to create a new Ticker object. This function accepts a `time.Duration` parameter `d` (the interval).

Here is an example:

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    ticker := time.NewTicker(time.Second)
```

```
    defer ticker.Stop()

    timeout, cancelFunc := context.WithTimeout(context.Background(), time.Second
    defer cancelFunc()

    go func() {
        for {
            select {
            case <-timeout.Done():
                fmt.Println("timeout done")
                return
            case <-ticker.C:
                fmt.Println("ticker fired!")
            }
        }
    }()

    // Main goroutine waits for 7 seconds to ensure we see the timer outputs
    time.Sleep(time.Second * 7)
}
```

The output of the code is as follows:

```
ticker fired!
ticker fired!
ticker fired!
ticker fired!
ticker fired!
timeout done
```

Step-by-step explanation:

1. Create a timer that fires every second. To ensure the timer is cleaned up at the end of the function, we use `defer ticker.Stop()`.

2. Create a context that times out after 5 seconds. `cancelFunc` is used to clean up the context before exiting.
3. In a new goroutine, a `select` statement listens to two channels: the timer's channel (`ticker.C`) and the context's done channel (`timeout.Done()`). When the timer fires each second, it prints a message. When the context times out (after 5 seconds), it prints a timeout message and returns, ending the goroutine.
4. The main goroutine uses `time.Sleep(time.Second * 7)` to wait 7 seconds, ensuring that both the timer firing and timeout events can be observed.

In addition to listening to `ticker.C` with `select`, you can also use a `for range` loop:

```
for range ticker.C {}
```


`ticker.C`, you need another mechanism to exit the loop, such as using a context.

Method Details

Reset

The `Reset(d Duration)` method is used to stop the ticker and reset its period to the specified duration. The next tick will occur after the new period has elapsed. It accepts a parameter `d` of type `time.Duration`, which represents

the new interval. This parameter must be greater than zero; otherwise, the `Reset` method will panic internally.

Here is an example:

```
package main

import (
    "time"
)

func main() {
    ticker := time.NewTicker(5 * time.Second)
    defer ticker.Stop()

    // Reset the ticker
    ticker.Reset(1 * time.Second)
    second := time.Now().Second()
    for t := range ticker.C {
        // 1s
        fmt.Printf("Interval: %d seconds", t.Second()-second)
        break
    }
}
```

The output of the code is as follows:

```
Interval: 1 seconds
```

Step-by-step explanation:

1. Create a time.Ticker that fires every 5 seconds.
2. Use the `Reset` method to change the interval from 5 seconds to 1 second.
3. In a single loop, print out the interval. The expected result is 1 second.

Stop

The `Stop()` method is used to stop the ticker. After calling `Stop`, no more ticks will be sent to channel `C`. Note: the `Stop` operation does not close the channel `C`.

Here is an example:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ticker := time.NewTicker(time.Second)
    quit := make(chan struct{}) // Create a quit channel

    go func() {
        for {
            select {
            case <-ticker.C:
                fmt.Println("Ticker fired!")
            case <-quit:
                fmt.Println("Goroutine stopped!")
                return // Exit the loop when receiving the quit signal
            }
        }
    }()

    time.Sleep(time.Second * 3)
    ticker.Stop() // Stop the ticker
```

```
        close(quit)   // Send the quit signal
        fmt.Println("Ticker stopped!")
    }
```

The output is as follows:

```
Ticker fired!
Ticker fired!
Ticker fired!
Goroutine stopped!
Ticker stopped!
```

1. Create a time.Ticker object that fires every second. At the same time, a quit channel of type `chan struct{}` is introduced, which is used to send a stop signal to the running goroutine.
2. Start a new goroutine. In this goroutine, a for-select loop listens to two events: ticker firing (`case <-ticker.C`) and the quit signal (`case <-quit`). Each time the ticker fires, it prints a message. If it receives the quit signal, it prints a message and exits the loop.
3. In the main goroutine, `time.Sleep(time.Second * 3)` simulates a waiting time of 3 seconds, during which the ticker will fire a few times.
4. The main goroutine stops the ticker by calling `Stop` , then closes the quit channel. The goroutine receives the quit signal, prints a message, and exits the loop.

The `Stop` method does not close channel `C` , so we need to use other means (such as a quit signal) to clean up resources.

Main Differences Between Timer and Ticker

Usage:

- Timer is used for tasks that are executed after a single delay.
- Ticker is used for tasks that need to be executed repeatedly.

Behavioral Characteristics:

- Timer fires once after the specified delay, sending a single time value to its channel.
- Ticker fires periodically at the specified interval, sending repeated time values to its channel.

Controllability:

- Timer can be reset (`Reset` method) and stopped (`Stop` method). `Reset` is used to change the firing time of the Timer.
- Ticker can also be reset (`Reset` method) and stopped (`Stop` method). `Reset` is used to change the interval at which the Ticker fires.

Termination Operation:

- The `Stop` method of Timer is used to prevent the Timer from firing. If the Timer has already fired, `Stop` does not remove the time value that has already been sent to its channel.
- The `Stop` method of Ticker is used to stop the periodic firing. Once stopped, no new values will be sent to its channel.

Notes

- For both Timer and Ticker, calling the `Stop` method does not close their `C` channels. If there are other goroutines listening on this channel, to avoid potential memory leaks, you need to manually terminate those goroutines. Usually, such resource cleanup can be handled by using a `context` or by a quit signal (implemented with channels).
- After a Ticker has completed its task, you should call the `Stop` method to release the associated resources and prevent memory leaks. If you do not stop the Ticker in time, it may result in continuous resource occupation.

Summary

This article has explored Go's Timer and Ticker in depth, introducing how to create them, their basic usage, and their related methods in detail. Additionally, the article summarizes the main differences between these two types of timers and emphasizes the considerations to keep in mind when using them.

When writing Go code, you should choose the appropriate timer according to the application scenario. At the same time, it's important to follow best practices — especially to release resources promptly after finishing with a timer — which is crucial for avoiding potential memory leaks.

We are Leapcell, your top choice for hosting Go projects.

Leapcell is the Next-Gen Serverless Platform for Web Hosting, Async Tasks, and Redis:

Multi-Language Support

- Develop with Node.js, Python, Go, or Rust.

Deploy unlimited projects for free

- pay only for usage — no requests, no charges.

Unbeatable Cost Efficiency

- Pay-as-you-go with no idle charges.
- Example: $25 supports 6.94M requests at a 60ms average response time.

Streamlined Developer Experience

- Intuitive UI for effortless setup.
- Fully automated CI/CD pipelines and GitOps integration.
- Real-time metrics and logging for actionable insights.

Effortless Scalability and High Performance

- Auto-scaling to handle high concurrency with ease.
- Zero operational overhead — just focus on building.

Explore more in the Documentation!

Follow us on X: @LeapcellHQ

Read on our blog

Web Development Programming Backend Go

Written by Leapcell

494 followers · 1 following

leapcell.io , web hosting / async task / redis

Follow

No responses yet

Write a response

What are your thoughts?

More from Leapcell

Leapcell

FastAPI at Lightning Speed ⚡: 10 Full-Stack Optimization Tips

10 FastAPI Performance Optimization Tips: End-to-End Speedup from Code to...

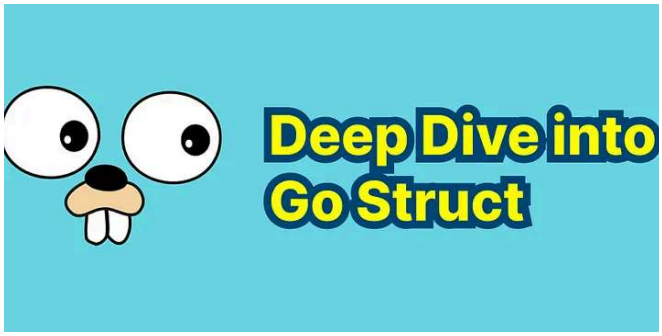
Aug 29, 2025 69 1

Leapcell

FastAPI + Uvicorn = Blazing Speed: The Tech Behind the Hype

What is Uvicorn?

Jan 10, 2025 13



Leapcell

Go Struct: A Deep Dive

Let's dive into all aspects of Go structs.

Jan 3, 2025 5 3

Leapcell

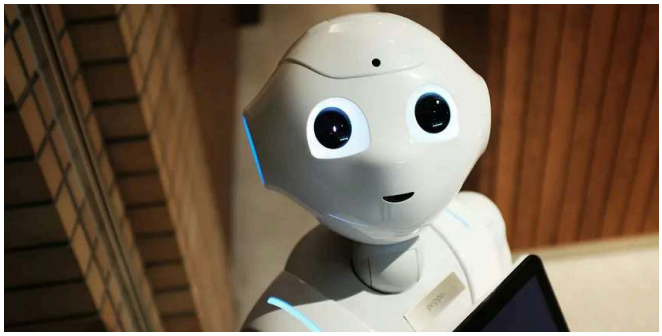
Simplifying Rust Error Handling with anyhow

A practical guide to simplifying Rust error handling using the anyhow crate.

May 1, 2025 102 2

See all from Leapcell

Recommended from Medium



Ade Mawan

Go 1.26 is Coming: What RC1 Tells Us

Release candidates are where Go's next release stops being "interesting in theory"...

Dec 31, 2025 110

Anshu Singhal

🔥 My Top 10 Golang Tricks for Super-Efficient Programming

You know, if you've been coding in Go for a bit, you totally get the whole simplicity thing. It's...

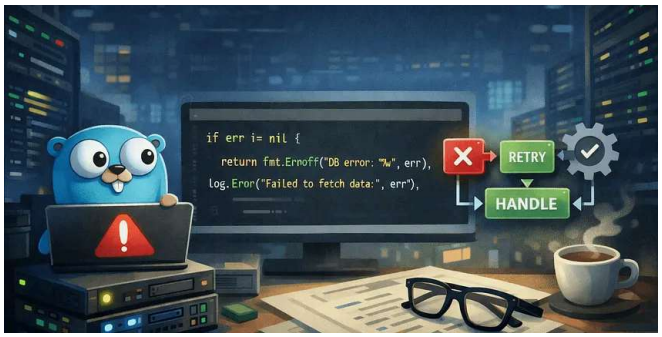
Dec 25, 2025 28

The Cache Cowgirl

This Tiny Golang App Pays My Rent

I never expected a 300-line Go program to become my side-income engine. But here w...

Jul 27, 2025 1.1K 22

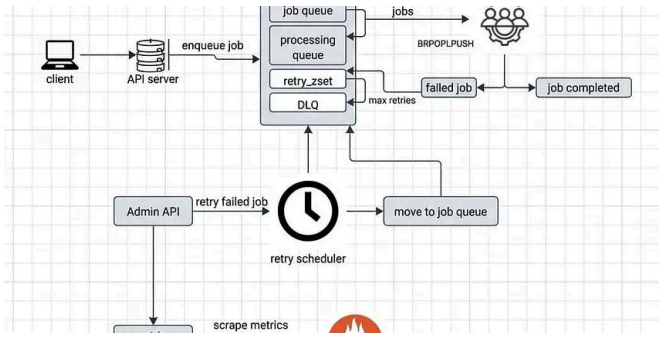


In Stackademic by Yaninyz witty

Error Handling in Go: Best Practices for Clean Code

Clean error handling used in production go services

Jan 6 68 2



Pushtisonawala

Designing a Production-Ready Background Job System in Golang

The reason why modern applications seem so fast is because they don't try to do all of their...

Cloud With Azeem

Go Libraries That Changed How We Code Forever (2026 Edition)

Remember when Go developers used to flex about "simplicity"? Yeah, that was before...

Jan 5 13 5 Oct 4, 2025 73 7

See more recommendations